MEDNET

Emergency Packet Prioritization Using a Synthetic Traffic Dataset

Computer Networks Project Review 3

Final Report and Local Simulation Product

Team member	Registration number
C SANJAY KARTHIK	24BCE2108
SUMEDHA P TUBAKAD	24BCE2075
DHEEREJ	24BCE2021
SATHYASEELAN N	24BCT0313
STEVE SUNNY SUBHASH	24BCE0354

Python simulation, reproducible CSV data,
and an offline local dashboard

1 Project Overview

Abstract

MedNet is a local Python product for evaluating packet scheduling on synthetic traffic. A saved reference dataset contains 4,750 packet records for six hospital-inspired classes across normal and congested conditions. The simulation compares first-in, first-out service, fixed priority, and a dynamic heuristic combining class priority, normalized deadline urgency, and queue occupancy. It models one bandwidth-limited, non-preemptive link with a finite waiting buffer. A browser dashboard exposes generation, dataset replay, parameter controls, policy comparisons, and downloadable results. Measurements include completion delay, buffer loss, throughput, link utilization, and on-time delivery among all generated packets. Repeated-seed experiments extend the reference run to examine variability. The product is a reproducible experimental tool; its statistical traffic settings are not calibrated to hospital captures or clinical requirements.

Problem and Objectives

A shared link must decide which waiting packet to transmit next. Different choices can reduce the delay of an urgent class while increasing the delay of another. The project asks whether the dynamic terms add useful behavior beyond a fixed-priority baseline, and measures the cost to lower-priority traffic on common input.

Objective	Delivered component
Specify the input	Seeded generation, saved CSV schema, and deterministic dataset replay.
Compare service policies	FIFO, fixed priority, and a documented dynamic scoring rule.
Make experiments accessible	Local dashboard with controls, results, and export.
Assess behavior transparently	Per-class outcomes, repeated-seed summaries, and automated checks.

Scope of the Completed Product

The completed product is the simulation, dataset workflow, local interface, and evaluation described in this report. It does not configure routers, transmit medical measurements, inspect real traffic, or establish clinical suitability. The hospital-inspired class names provide a motivating context; every input row is generated metadata.

2 Literature: Synthetic Traffic and Evaluation

Reproducible Workloads

Botta, Dainotti, and Pescapé [1] describe D-ITG, including statistical workload specification, packet-size and inter-packet-time distributions, and random seeds. This supports recording generator settings and retaining common inputs. MedNet uses its own Python generator and does not reproduce D-ITG or inherit its validation.

Limitations of Poisson Models

Paxson and Floyd [2] examine measured wide-area traffic and distinguish session arrivals from packet arrivals. For the traffic studied, exponential packet interarrivals could underestimate burstiness even when some user-initiated session arrivals were compatible with a Poisson model. MedNet therefore treats exponential interarrivals as an explicit controlled workload assumption. The paper does not establish whether these six generated classes resemble hospital traffic.

Network Simulation Methodology

Floyd and Paxson [3] discuss implementation verification, reproducibility, sensitivity to scenario choices, and the limits of fixed-trace replay when real sources would respond to network conditions. These concerns motivate the saved traces, policy comparisons, repeated seeds, and explicit limitations in this project. The current generator is open loop: its arrivals do not slow down when a queue becomes crowded.

Literature principle	Application in MedNet
Describe the workload precisely [1]	Record seed, rate, size distribution, horizon, and deadlines.
Avoid unsupported realism claims [2]	Label data synthetic and keep rates separate from clinical claims.
Check dependence on experiments [3]	Reuse each trace and report variation across repeated seeds.
Verify the implemented model [3]	Check packet timing, admission, ordering, and accounting.

The papers support methodology and interpretation. They are not the source of the configured numerical deadlines, arrival rates, priorities, or score coefficients.

3 Literature: Packet Prioritization

Healthcare Communication Context

Bhandari and Moh [4] evaluate PA-MAC for wireless body area networks and use Poisson arrivals in their experiment. Their wireless MAC model and payload settings differ from the single-link model here. The reference supports the use of modeled traffic in a healthcare communication study, rather than validating this dataset.

Waiting Time and Congestion

Mazloomi, Gholipour, and Zaretalab [5] combine traffic differentiation with scheduling, routing, and rate control. Their scheduling discussion considers elapsed waiting time. MedNet also considers waiting time, but uses a different bounded urgency equation and occupancy boost. It does not implement the paper's two-queue scheduling equation, routing, or machine-learning components.

Delay and Packet Size

Asingwire et al. [6] present an analytical model with Poisson arrivals, heterogeneous servers, and infinite queue capacity. Their comparison with EDF and Priority EDF is relevant to examining urgency and fairness. MedNet uses a finite queue and one link; packet size determines serialization time, but the scheduling score does not favor smaller packets.

Deadlines and Guarantees

Liebeherr, Wrege, and Ferrari [7] compare EDF, static-priority scheduling, and rotating-priority queues in a bounded-delay setting. Admission control and traffic characterization are central to their guarantees. MedNet's weighted score is not EDF, and its finite-buffer admission rule does not provide a bounded-delay guarantee.

Position of This Project

The contribution is a reproducible synthetic workload, an accessible implementation, and an empirical comparison of three service policies. Priority and deadline-based scheduling are established ideas. The project does not claim a new scheduling theory or reproduce a complete algorithm from one base paper. Fixed priority is retained so that benefits due to class labels can be separated from those due to dynamic scoring.

4 Dataset and Provenance

The reference file `results/trace.csv` contains 4,750 records. Each row represents a packet offered to the model, including packets later rejected by the waiting buffer. Identifiers restart within a scenario; the pair `(scenario, packet_id)` identifies a record.

Field	Meaning
scenario	normal or congested; selects the input scenario.
packet_id	Nonnegative integer, unique within a scenario.
kind	emergency, monitoring, video, images, records, or admin.
arrival_s	Arrival time in seconds from scenario time zero.
size_bytes	Positive integer length used for serialization.
deadline_s	Absolute completion deadline in seconds from time zero.

Table 1. Input schema used by generation and replay.

Class	Normal	Congested	Total
Emergency	49	127	176
Monitoring	231	583	814
Video	630	1534	2164
Images	293	776	1069
Records	87	245	332
Admin	65	130	195
Total	1355	3395	4750

Table 2. Exact counts of the saved seed-42 reference dataset.

Rows contain network metadata, not patient records, diagnoses, ECG samples, or image pixels. The dataset is neither a packet capture nor a trained generative model. No machine-learning train/test split is involved: each policy receives every row of the selected scenario.

SHA-256 fingerprint of the saved reference CSV:

```
472053455d80dae3172aa8f77125210d9e213cea4da9b864b78c110d1dbcb333
```

5 Product Architecture

The local browser interface and the command-line workflow call the same Python simulation core. Changing a control changes experiment settings; it does not alter the definitions of the three scheduling policies.

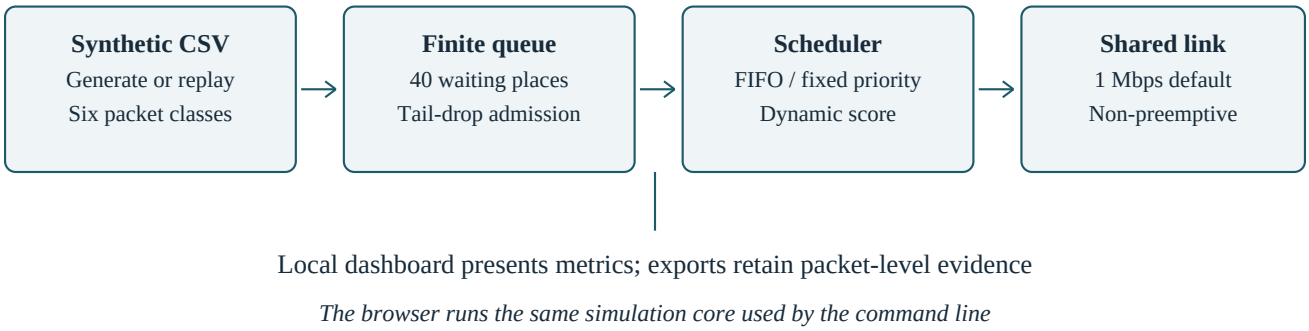


Figure 1. Input, service, and measurement flow. Values shown are defaults.

Layer	Responsibility
Local web interface	Collect settings, start generation or replay, display comparisons, and download outputs.
HTTP application (app.py)	Serve bundled assets and translate requests into simulation calls.
Simulation core (mednet.py)	Validate data, generate arrivals, admit packets, schedule service, and measure results.
Experiment outputs	Store input trace, packet outcomes, class metrics, and run configuration.
Repeated-seed evaluation	Run the unchanged policy comparison on additional independently seeded traces.

Local Operation

The application uses Python's standard-library HTTP server and bundled browser assets. A local URL opens the dashboard; no external account or hosted backend is required for the experiment. The browser is a control and results interface. Packet transmission occurs in virtual simulation time, not through network sockets to medical equipment.

Separation of Data and Outcomes

The input trace is generated or loaded before scheduling. Metrics and packet outcomes are outputs. Keeping these separate allows the same data to be replayed while changing a policy or a link parameter, and makes dropped packets visible in the evaluation.

6 Model, Parameters, and Measurements

Class	Priority	Bytes	Packets/s	Deadline ms
Emergency	5	256	3	80
Monitoring	5	512	12	150
Video	4	1200	32	200
Images	3	1400	16	600
Records	2	1000	5	1000
Admin	1	700	3	1500

Table 3. Generator settings for normal load; congested load multiplies each rate by 2.4.

The reference experiment uses a 1 Mbps link, 40 waiting positions, a 20-second arrival horizon, and seed 42. Interarrivals are exponentially distributed. Packet sizes are nominal sizes multiplied by a uniform draw from 0.8 to 1.2 and rounded to integer bytes. Deadlines are arrival time plus the configured relative interval. All values are experimental settings, not established hospital requirements.

One packet occupies the link for size_bytes x 8 / link_bps seconds. An arriving packet starts if the link is idle; otherwise it joins the waiting queue if space exists. A full queue rejects the new arrival, even if critical. Service cannot be interrupted. At equal timestamps, completion and selection from the existing queue precede new arrivals. Accepted packets drain after arrivals stop.

Measure	Definition
Waiting / completion delay	Start minus arrival / finish minus arrival; among delivered packets.
Buffer drops / delivered late	Refused arrivals / completed packets finishing after their deadline.
On-time success	On-time delivered packets divided by all generated packets, including drops.
Throughput	All delivered bits divided by the whole observation interval.
Link utilization	Total transmission time divided by the same observation interval.

The observation interval is zero to the later of the arrival horizon or final completion. It includes idle time and queue draining and can differ across policies. These are whole-run values, not steady-state estimates. Packet size is total modeled serialization length; separate headers, propagation, and retransmissions are omitted.

7 Implementation: Data Generation

The following excerpts show implemented components of the larger program. The complete source is provided with the product; this report does not print the complete application.

```
@dataclass(frozen=True)
class Packet:
    packet_id: int
    kind: str
    arrival_s: float
    size_bytes: int
    deadline_s: float

def generate_trace(horizon_s, seed, load=1.0):
    rng = random.Random(seed)
    trace = []
    for kind, spec in CLASSES.items():
        time_s = 0.0
        rate = spec.rate_per_s * load
        if rate <= 0:
            continue
        while True:
            time_s += rng.expovariate(rate)
            if time_s >= horizon_s:
                break
            size = max(1, round(
                spec.packet_bytes * rng.uniform(0.8, 1.2)
            ))
            deadline = time_s + spec.deadline_ms / 1000
            trace.append((time_s, kind, size, deadline))
    trace.sort()
    return [Packet(i, kind, arrival, size, deadline)
            for i, (arrival, kind, size, deadline)
            in enumerate(trace)]
```

Listing 1. Packet model and generator logic. Input validation is omitted from this excerpt; formatting is adjusted for the report.

Repeatability and Replay

The generator uses a local random-number instance initialized from the seed. A generated scenario is shared unchanged by the three policies. Normal and congested traces are generated separately; the same seed does not make them simple time-scaled copies because different numbers of random draws are consumed.

CSV replay parses the saved metadata instead of drawing new arrivals. Validation checks the schema, supported labels, times, positive sizes, and duplicate identifiers. Replaying a fixed trace preserves its input; it does not model a source reacting to congestion.

```
python mednet.py --output results
python mednet.py --input-dataset results/trace.csv --output replay_results
```

8 Implementation: Dynamic Packet Selection

When the link is free, FIFO chooses by arrival order and fixed priority chooses by class priority. MedNet recomputes a score for every waiting packet. Its urgency is elapsed deadline-window fraction, capped at one. Critical traffic gains up to 0.75 additional points as queue occupancy rises from 75% to 100%.

```
def dynamic_score(packet, now_s, occupancy, score):
    window = packet.deadline_s - packet.arrival_s
    urgency = 1.0 if window <= 0 else min(
        1.0, max(0.0, (now_s - packet.arrival_s) / window)
    )
    congestion = min(1.0, max(
        0.0,
        (occupancy - score.congestion_threshold)
        / (1.0 - score.congestion_threshold)
    ))
    boost = (score.critical_boost * congestion
             if packet.kind in CRITICAL else 0.0)
    return (CLASSES[packet.kind].priority
            + score.urgency_weight * urgency + boost)

def choose_packet(waiting, policy, now_s, capacity, score):
    def key(index):
        packet = waiting[index]
        value = 0.0
        if policy == "priority":
            value = CLASSES[packet.kind].priority
        elif policy == "mednet":
            value = dynamic_score(
                packet, now_s, len(waiting) / capacity, score
            )
        return (-value, packet.arrival_s, packet.packet_id)
    return min(range(len(waiting)), key=key)
```

Listing 2. Score and selection logic. Default urgency_weight is 3.0.

Urgency applies to every class. Ties use arrival time and then packet identifier; the negative score makes the largest value win. Expired packets remain eligible. Selection is called only when the waiting queue is nonempty; a zero-capacity queue cannot reach that call.

This heuristic is not EDF and does not calculate transmission-time slack. Its weights are configured choices rather than fitted or proven optimal values. Recalculation is essential: storing a score only on arrival would discard the intended time dependence. Bounded urgency does not guarantee freedom from starvation.

9 Local Dashboard and Operating Workflow

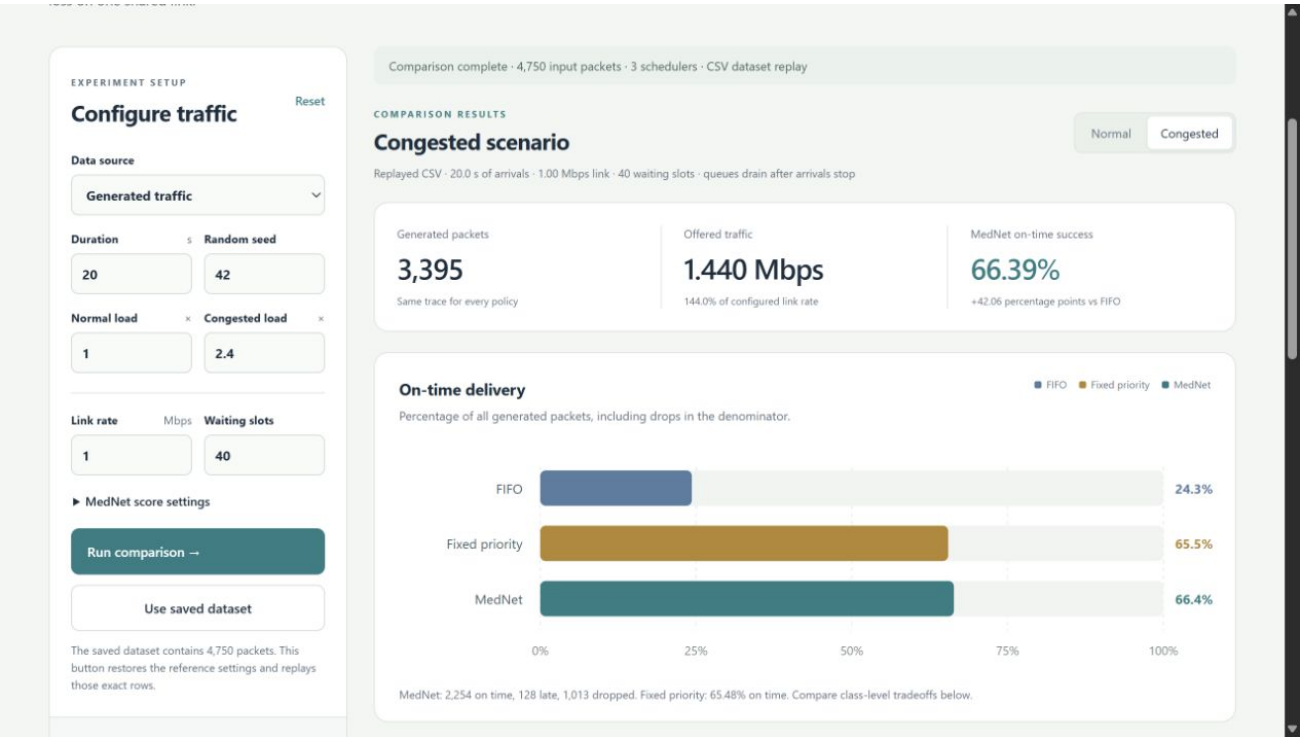


Figure 2. Working local dashboard displaying a simulation comparison.

Start and Run

```
python app.py
```

Extract the product ZIP and use Python 3.10 or newer. On Windows, double-click start_mednet.bat, or run the command above from the extracted folder. The app opens `http://127.0.0.1:8765`. No third-party Python packages are needed. Choose generated traffic or CSV replay, configure the experiment, then click Run comparison. All three policies run together.

Step	Purpose
1. Choose data and settings	Generate a workload or replay CSV; set the exposed link and queue controls.
2. Run the comparison	Evaluate all three policies using the same input within each scenario.
3. Read class-level results	Compare timeliness together with drops and routine-traffic delay.
4. Download outputs	Retain the result archive for inspection and repeatability.

The dashboard shows the saved dataset on opening. Use saved dataset restores the reference settings. The first 100 arrivals per scenario/policy can be inspected; the ZIP contains all outcomes. Download results before stopping Python: eight recent runs are cached in memory. Simulation time advances directly between events, so a 20-second arrival horizon is not a real-time wait.

10 Reference Dataset Results

The reference experiment uses seed 42 and the settings in Table 3. In the saved normal scenario, all 1,355 packets were delivered before their configured deadlines by every policy. Under congestion, the same 3,395 inputs yielded the following results.

Congested scenario	FIFO	Fixed priority	MedNet
Delivered packets	2385	2377	2382
Buffer drops	1010	1018	1013
Delivered late	1559	154	128
On-time success (%)	24.33	65.48	66.39
Overall mean delay (ms)	330.50	330.26	330.47
Throughput (Mbps)	0.9990	0.9990	0.9990
Link utilization (%)	99.90	99.90	99.90

Table 4. Whole-run outputs for the reference congested trace.

Mean delay (ms)	FIFO	Fixed priority	MedNet
Emergency	326.05	7.31	6.97
Monitoring	328.46	9.74	9.95
Video	331.97	20.91	55.50
Images	329.19	98.34	326.53
Records	333.79	833.68	926.65
Admin	328.49	6683.20	4148.61

Table 5. Completion delay among delivered packets only.

Most of the improvement over FIFO was already achieved by fixed priority. MedNet increased on-time success by 0.91 percentage points beyond fixed priority in this run, while video, monitoring, images, and records had higher mean delay. Administrative delay remained much larger than under FIFO.

MedNet dropped 32 of 127 emergency packets. Low delay among its delivered emergency packets therefore does not mean guaranteed delivery. Service order can change when queue space opens, so policies can admit different subsets of the same input.

11 Repeated-Seed Evaluation

The benchmark uses seeds 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 and load multipliers 1.0, 1.8, 2.4, 3.0: 40 shared traces and 120 policy simulations. The link remains 1 Mbps, waiting capacity 40, arrival duration 20 seconds, and scoring settings unchanged. Each seed/load trace is reused by all three policies. Seed 42 is retained separately for the reference demonstration.

Values below are unweighted means +/- sample standard deviations across 10 seeds. SD describes variation within this model, not a confidence interval or uncertainty about workload realism.

Load multiplier	FIFO	Fixed priority	MedNet
1.0	100.00 +/- 0.00	100.00 +/- 0.00	100.00 +/- 0.00
1.8	41.61 +/- 7.11	88.52 +/- 2.10	89.90 +/- 2.32
2.4	25.20 +/- 0.79	65.67 +/- 1.11	66.77 +/- 1.18
3.0	19.92 +/- 0.70	52.45 +/- 1.00	53.66 +/- 0.85

Table 6. On-time success percent, mean +/- SD; drops remain in the denominator.

Delay at load 2.4 (ms)	FIFO	Fixed priority	MedNet
Emergency	316.66 +/- 5.33	7.58 +/- 0.40	7.64 +/- 0.30
Monitoring	319.54 +/- 4.16	9.65 +/- 0.18	9.86 +/- 0.19
Video	325.17 +/- 2.56	20.67 +/- 0.84	47.00 +/- 3.58
Images	326.95 +/- 3.61	100.15 +/- 15.92	303.69 +/- 10.67
Records	321.86 +/- 6.35	633.34 +/- 138.00	842.15 +/- 36.57
Admin	320.01 +/- 6.92	5548.55 +/- 454.52	3982.70 +/- 346.33

Table 7. Across-seed mean and SD of each run's delivered-packet mean delay.

At load 2.4, paired MedNet minus fixed-priority on-time success is +1.095 percentage points, SD 0.412. MedNet improves aggregate on-time delivery but increases mean video, image, record, emergency, and monitoring delay relative to fixed priority; administrative delay decreases. Tail drops remain possible for critical packets.

The experiment varies load and seed. It does not establish superiority for arbitrary traffic or validate the Poisson model. Bandwidth, buffer capacity, and scoring sensitivity remain separate evaluation dimensions.

12 Verification and Reproducibility

Checks Performed

28 automated tests passed in the final verification run. The tests check the implementation against concrete expectations; they do not establish realism of the traffic model or optimality of the heuristic.

Area	Expected property
Timing and non-preemption	A packet occupies the link for its serialization time; an active transmission finishes before reselection.
Scheduling and ties	Urgency and congestion can alter selection; ties resolve deterministically.
Finite-buffer behavior	Full queues reject arrivals without evicting already admitted packets.
Metric conservation	Generated equals delivered plus dropped; delivered equals on-time plus late.
Data loading and replay	Valid CSV loads consistently; malformed data is rejected; the saved input reproduces expected outputs.
Interface and download	Dashboard controls complete a run and expose the corresponding saved results.

The browser was checked with the reference dataset, both scenario views, a 2 Mbps link run, CSV upload and replay, per-class loss selection, and a downloaded result ZIP. The reference CSV outputs match Review 2 byte for byte. The 28 tests comprise 15 core checks, 5 repeated-experiment checks, and 8 dashboard checks.

Reproduce an Experiment

```
python app.py
python mednet.py --output results
python mednet.py --input-dataset results/trace.csv --output replay_results
python experiment.py --output experiments_new
python -m unittest -v
```

Run commands from the product directory. Preserve the input trace and configuration with each result. A new run with changed input settings is a different experiment; a CSV replay retains its packet arrivals and sizes. In a replay, the original generator seed is provenance, not a command to redraw packets.

Evidence Files

`trace.csv` records input; `outcomes.csv` records every delivery or drop; `metrics.csv` records class and total summaries; `config.json` records experiment settings. The benchmark adds the seed dimension so repetitions remain distinguishable.

13 Limitations and Conclusion

Limits of the Model

Only one shared link is modeled. There is no routing, propagation, interference, transport feedback, retransmission, or medical device. The six labels describe synthetic classes, and their rates, packet sizes, deadlines, and priorities have not been calibrated to measured hospital traffic. Independent exponential arrivals do not establish realistic burstiness [2]. Repeated seeds address sampling variation within this model, not the validity of the model itself.

Tail-drop admission can discard critical packets. Non-preemptive service can delay an urgent arrival behind an ongoing transmission. Bounded urgency does not guarantee fairness or prevent indefinite starvation under sustained load. The score is neither EDF nor a reservation scheme. Its coefficients have not been optimized or given formal delay guarantees.

Metrics must retain their denominators. Delay statistics exclude dropped packets, while on-time success includes every offered packet. Queue draining ensures complete accounting but does not demonstrate steady-state performance. When no packet is delivered, numerical delay placeholders should be read alongside the delivery count.

Further Work

Further evaluation can extend the tested load range and vary bandwidth, buffer capacity, and score weights. Additional methods could include an EDF baseline, weighted service, or stronger aging for lower-priority classes. Alternative arrival models or measured traces would support a separate assessment of realism. Admission policies protecting critical traffic should be tested independently from service ordering. Multi-link simulation would require additional routing and delay assumptions.

Conclusion

MedNet delivers a working local workflow from synthetic data generation or replay to scheduling, measurement, and export. The interface makes the experiment accessible while preserving its input and outputs for inspection. The reference run shows that fixed priority accounts for most of the improvement in on-time completion over FIFO, while dynamic scoring produces smaller, workload-dependent changes and redistributes delay among classes. Repeated-seed evaluation makes variation visible without claiming clinical validation. The product demonstrates computer-network concepts through a defined, reproducible model rather than a deployed hospital system.

14 References

[1] A. Botta, A. Dainotti, and A. Pescapé, "A tool for the generation of realistic network workload for emerging networking scenarios," *Computer Networks*, vol. 56, no. 15, pp. 3531-3547, 2012. DOI: 10.1016/j.comnet.2012.02.019.
Publisher record

[2] V. Paxson and S. Floyd, "Wide area traffic: the failure of Poisson modeling," *IEEE/ACM Transactions on Networking*, vol. 3, no. 3, pp. 226-244, 1995. DOI: 10.1109/90.392383.
Author full text

[3] S. Floyd and V. Paxson, "Difficulties in Simulating the Internet," *IEEE/ACM Transactions on Networking*, vol. 9, no. 4, pp. 392-403, 2001. DOI: 10.1109/90.944338.
Author full text

[4] S. Bhandari and S. Moh, "A Priority-Based Adaptive MAC Protocol for Wireless Body Area Networks," *Sensors*, vol. 16, no. 3, article 401, 2016. DOI: 10.3390/s16030401.
Open full text

[5] N. Mazloomi, M. Gholipour, and A. Zaretalab, "A priority-based congestion avoidance scheme for healthcare wireless sensor networks," *IET Wireless Sensor Systems*, vol. 13, no. 1, pp. 9-23, 2023. DOI: 10.1049/wss2.12046.
Publisher full text

[6] B. K. Asingwire, L. Sibomana, A. Ngenzi, and C. Kabiri, "Delay and size-dependent priority-aware scheduling for IoT-based healthcare traffic using heterogeneous multi-server priority queueing system," *IET Communications*, vol. 17, no. 16, pp. 1877-1887, 2023. DOI: 10.1049/cmu2.12675.
Publisher full text

[7] J. Liebeherr, D. E. Wrege, and D. Ferrari, "Exact Admission Control for Networks with a Bounded Delay Service," *IEEE/ACM Transactions on Networking*, vol. 4, no. 6, pp. 885-901, 1996. DOI: 10.1109/90.556345.
Author full text

Reference [5] appeared online in December 2022 and in the 2023 journal issue. These references support the literature discussion. Dataset counts, verification evidence, and reported experiment results come from the MedNet product files.